

DEADLINE: TBA

YOU SHOULD PROVIDE: A report in a .pdf file and a zipped `project1_s123456.zip` (your whole project). The report should be understandable even to someone who did not attend the lecture; it should include not only your results but also a clear statement of the problem and explanations of your solution.

The project may be completed individually or in groups of up to two persons.

REMARK: Below you will find the elements that should be included in your project. Your project should contain all these components, although you are free to add additional features.

Recommender System

Objective

Implement a recommendation system for movie ratings using various algorithms. In this project, you are expected to follow good Python practices, including modular code organization, the proper use of command-line argument parsing (via `argparse`), and a clear project structure. Your project will be evaluated based on both the quality of your report (in PDF format) and the performance of your implementation (measured by RMSE) on the provided test data. The implementation should support both training a model and using a saved model for prediction.

Data

Download the file `ml-latest-small.zip` from <https://grouplens.org/datasets/movielens/>. Extract the file `ratings.csv` (we are interested only in this file). The dataset contains approximately 100,000 ratings – about 600 users have rated around 9,000 movies. Its format is as follows:

```
userId,movieId,rating,timestamp
1,1,4.0,964982703
1,3,4.0,964981247
1,6,4.0,964982224
1,47,5.0,964983815
1,50,5.0,964982931
...
```

Here, `userId` is a unique user identifier, `movieId` is a unique movie identifier, and `rating` is a rating from 0 to 5 (ratings may also include half points, e.g., 3.5, 4.5). The `timestamp` field will not be used.

It may be convenient to convert the data into matrix form. Let $\mathbf{Z}$ be a matrix containing the training ratings – a matrix of size $n \times d$, where n is the number of users and d is the number of movies. Thus, the sample fragment of `ratings.csv` (after applying an appropriate `userId`-to-integer mapping, and recalling that Python indexing starts at 0) might be converted to:

$$\mathbf{Z}[0,0] = 4.0, \mathbf{Z}[0,2] = 4.0, \mathbf{Z}[0,5] = 4.0, \mathbf{Z}[0,46] = 5.0, \mathbf{Z}[0,49] = 5.0, \dots$$

Note that $\mathbf{Z}$ is *sparse* – many entries of $\mathbf{Z}$ are missing (i.e., unrated).

Task

Train five models – NMF, SVD1, SVD2, SGD (described below) and BEST – using the data from `ratings.csv`. Your implementation should be capable of training on this type of dataset (and may be tested on other datasets of the same form), and it must also be able to predict ratings for unseen user–movie pairs. **All predicted ratings must be rounded to the nearest 0.5.**

Project Structure

Your zipped project must have the following minimum structure. The root folder must be named exactly `project1_s123456`, where `s123456` is your student number. A recommended structure is as follows:

```
project1_s123456/
|-- data/                % Input data files (e.g., ratings.csv)
|-- modules/             % Python modules
|   |-- utils.py         % Utility functions
|   |-- train.py         % Training-related functions
|   |-- predict.py       % Prediction-related functions
|-- models_trained/      % Saved trained models
|-- results/             % Prediction outputs
|-- README.md            % Project documentation
'-- main.py              % Main entry point (uses argparse)
```

Sample Project

A sample project, together with, e.g., an evaluation tool and sample test sets, is available in the file `sample_project1.zip`. Its structure is as follows:

```
sample_project1/
|-- project1_s123456/    % Sample student project
|-- tools/              % Tools, e.g., evaluation script
```

```
|-- sample_test.csv           % Sample file for prediction
'-- sample_test_with_ratings.csv % Corresponding true ratings
                                % (of course, won't be provided)
```

It already includes a simple implementation of NMF.

main.py requirements

- **Entry Point:** The main file must be named `main.py`.
- **Argument Parsing:** Use `argparse` to implement a function (e.g., `ParseArguments()`) that parses the following options:

- `--mode`: mode of operation; allowed values: `train`, `predict`.
- `--train_file`: the training data file (e.g., `ratings.csv`).
- `--input_file`: the input file for prediction (e.g., `sample_test.csv`).
- `--model_path`: where to save the trained model (in training mode) or from where to load it (in prediction mode),
e.g., `project1_s123456/models_trained/model_NMF.pkl`.
- `--output_file`: where to save predictions,
e.g., `project1_s123456/results/preds.csv`.
- `--alg`: algorithm to use (allowed values: `NMF`, `SVD1`, `SVD2`, `SGD`, `BEST`).

The examples below should work for the provided `sample_project1`.

- **Modes of Operation:**

1. **Training Mode:** Example command:

```
python project1_s123456/main.py --mode train
--train_file project1_s123456/data/ratings.csv
--model_path project1_s123456/models_trained/model_NMF.pkl
--alg NMF
```

In training mode, your script should load the training data, train the selected algorithm, and save the trained model.

2. **Prediction Mode:** Example command:

```
python project1_s123456/main.py --mode predict
--input_file sample_test.csv
--model_path project1_s123456/models_trained/model_NMF.pkl
--output_file project1_s123456/results/preds.csv
--alg NMF
```

In prediction mode, your script should load the stored model, generate predictions, and write them to the output file.

The output file `preds.csv` must have exactly the same format as in the provided `sample_project1`. Please check the sample project for the required column names and file structure. In particular, the output should contain the predicted ratings for all user–movie pairs listed in the input file for prediction.

Evaluation

Your project will be evaluated based on:

1. **Report Quality:** A PDF report explaining your approach, methodology, and design choices.
2. **Model Performance:** The performance of your system, measured by RMSE on test data.

Automated Evaluation: Later, a bash script will run your project in training mode to generate models (e.g., using a training file such as `ratings.csv`). Then, the script will run your project in prediction mode on several test files. The predictions will be compared with ground truth, and a **root-mean square error** will be computed:

$$\text{RMSE} = \sqrt{\frac{1}{|\mathcal{T}|} \sum_{(u,m) \in \mathcal{T}} (\mathbf{Z}'[u, m] - \mathbf{T}[u, m])^2}, \quad (1)$$

where $\mathcal{T}$ is a test set, $\mathbf{Z}'[u, m]$ is a rate prediction for user u and movie m , and $\mathbf{T}[u, m]$ is the “true” rating.

Evaluation tool is available in `tools/evaluate_solution.py`.

Assuming previous training and prediction was performed, the evaluation of the prediction is as follows:

Each algorithm will be evaluated on 10 different training–test splits, and the final RMSE will be computed as the average over all runs.

```
python tools/evaluate_solution.py
--true_file sample_test_with_ratings.csv
--pred_file project1_s123456/results/preds.csv
RMSE: 2.8583
```

Algorithm Details

NMF (Non-negative Matrix Factorization)

NMF approximates the matrix $\mathbf{Z}$ of size $n \times d$ as

$$\mathbf{Z} \approx \mathbf{WH},$$

where $\mathbf{W}$ is of size $n \times r$ and $\mathbf{H}$ is of size $r \times d$, with r as a parameter.

Implementation in Python:

```
from sklearn.decomposition import NMF
import numpy as np

model = NMF(n_components=r, init='random', random_state=0)
W = model.fit_transform(Z)
H = model.components_
Z_approximated = np.dot(W, H)
```

More details on NMF in scikit-learn: <https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.NMF.html>

This implementation requires a fully populated matrix $\mathbf{Z}$. Therefore, missing values must first be imputed. You should propose a method for handling missing values, such as:

- Filling missing entries with 0.
- Replacing missing values with the average rating of the corresponding movie.
- Replacing missing values with the average rating given by the user.

Your algorithm should also determine the BEST choice of r . In `sample_project1`, a simple NMF model is implemented with $r = 5$ (arbitrarily chosen) and missing values set to 0.

SVD1

This method follows the same approach as NMF, but instead of NMF, Singular Value Decomposition (SVD) is used.

SVD approximates $\mathbf{Z}$ as

$$\mathbf{Z} \approx \mathbf{U}_r \mathbf{\Lambda}_r \mathbf{V}_r^T,$$

where $\mathbf{U}_r$, $\mathbf{\Lambda}_r$, and $\mathbf{V}_r^T$ are obtained from the truncated SVD decomposition. To maintain consistency with NMF, we set $\mathbf{W} = \mathbf{U}_r$ and $\mathbf{H} = \mathbf{\Lambda}_r \mathbf{V}_r^T$.

Implementation in Python:

```
from sklearn.decomposition import TruncatedSVD
import numpy as np

svd = TruncatedSVD(n_components=r, random_state=42)
svd.fit(Z)
Sigma2 = np.diag(svd.singular_values_)
VT = svd.components_

W = svd.transform(Z) / svd.singular_values_
H = np.dot(Sigma2, VT)
```

More details: <https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.TruncatedSVD.html>

SVD2

This method follows a different approach for computing the SVD approximation. The procedure will be described during the lecture or lab sessions.

SGD (Stochastic Gradient Descent)

In the previous methods, missing values in the matrix $\mathbf{Z}$ had to be imputed before applying factorization. Here, we reformulate the problem so that missing values are handled directly in the optimization process.

For a given $\mathbf{Z}$ of size $n \times d$, we seek matrices $\mathbf{W}$ (size $n \times r$) and $\mathbf{H}$ (size $r \times d$) by solving:

$$\arg \min_{\mathbf{W}, \mathbf{H}} \sum_{(i,j): z_{ij} \neq '?'} (z_{ij} - \mathbf{w}_i^T \mathbf{h}_j)^2$$

or, with regularization ($\lambda > 0$ as a parameter),

$$\arg \min_{\mathbf{W}, \mathbf{H}} \sum_{(i,j): z_{ij} \neq '?'} (z_{ij} - \mathbf{w}_i^T \mathbf{h}_j)^2 + \lambda (||\mathbf{w}_i||^2 + ||\mathbf{h}_j||^2),$$

where $\mathbf{h}_j$ is the j -th column of $\mathbf{H}$, and $\mathbf{w}_i^T$ is the i -th row of $\mathbf{W}$.

Various optimization techniques can be used to minimize this objective function, which is a function of matrices $\mathbf{W}$ and $\mathbf{H}$. For this task, use `PyTorch` to implement the optimization procedure, as demonstrated in:

LIST NR 2: Gradient methods. Automatic differentiation. (available on GitHub¹)

You may use the Stochastic Gradient Descent (SGD) or Adam optimizer to find the BEST matrices $\mathbf{W}$ and $\mathbf{H}$. Additionally, the two loss functions provided above are just examples – you may define your own and optimize it accordingly. Once trained, the matrices $\mathbf{W}$ and $\mathbf{H}$ should be stored as the model output.

BEST

The BEST model should correspond to your BEST-performing approach. It may include any improvements, modifications, or combinations of the required methods (e.g., tuning, regularization, bias terms, or hybrid models). The goal is to achieve the lowest possible RMSE on the validation/test data.

¹https://github.com/lorek/MethodsClassDimRed/blob/main/MoCaDR_List_nr_2_auto_grad.ipynb